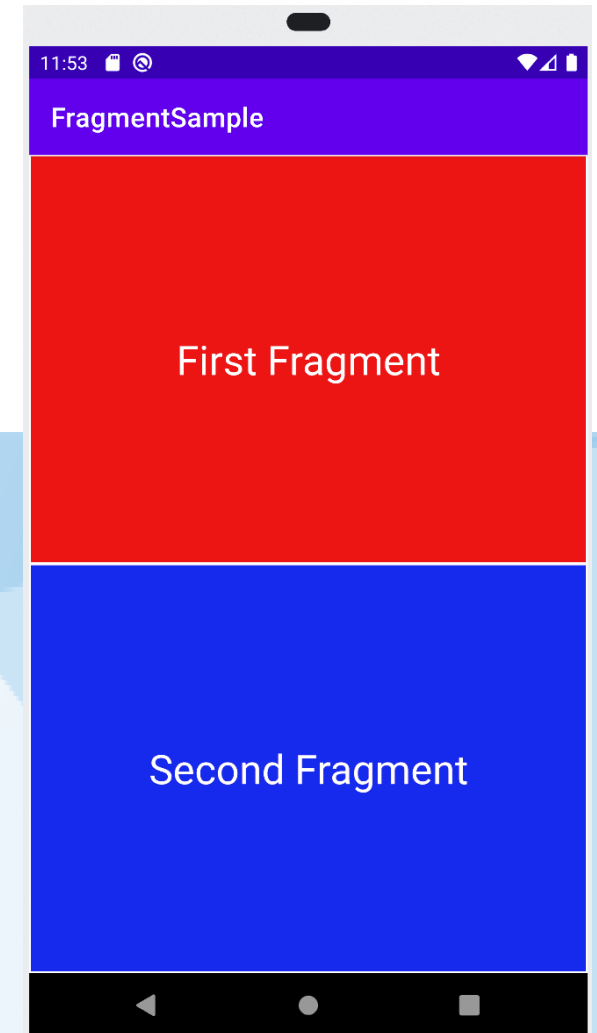


Lecture 5

Fragments

Course: Mobile App Development

By: Tho C. Mai
Nha Trang University



Objectives



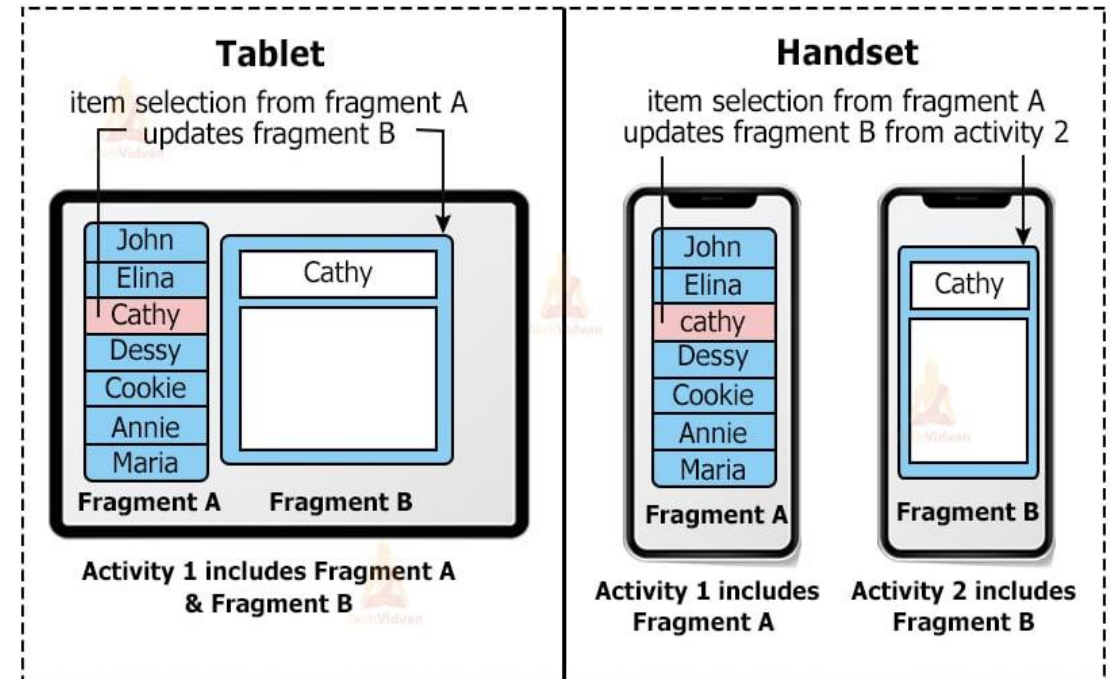
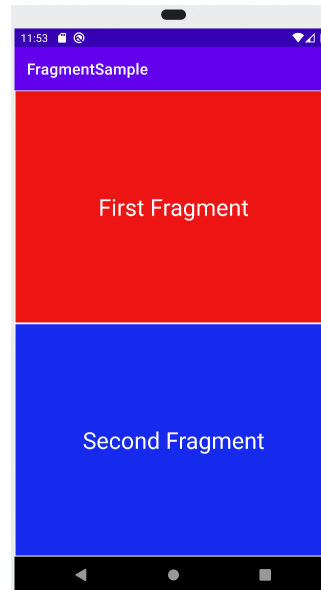
- Understand the concept and role of Fragments in Android.
- Learn how to create and use Fragments in an application.
- Understand communication between Fragments and Activities.
- Use Fragments to build flexible and reusable UI components.

What is a Fragment?

- An **Activity** is a container for views => typically fills the entire screen
- A **Fragment** is a modular section of an Activity's UI.
- It has its own lifecycle but is tightly linked to the Activity containing it.

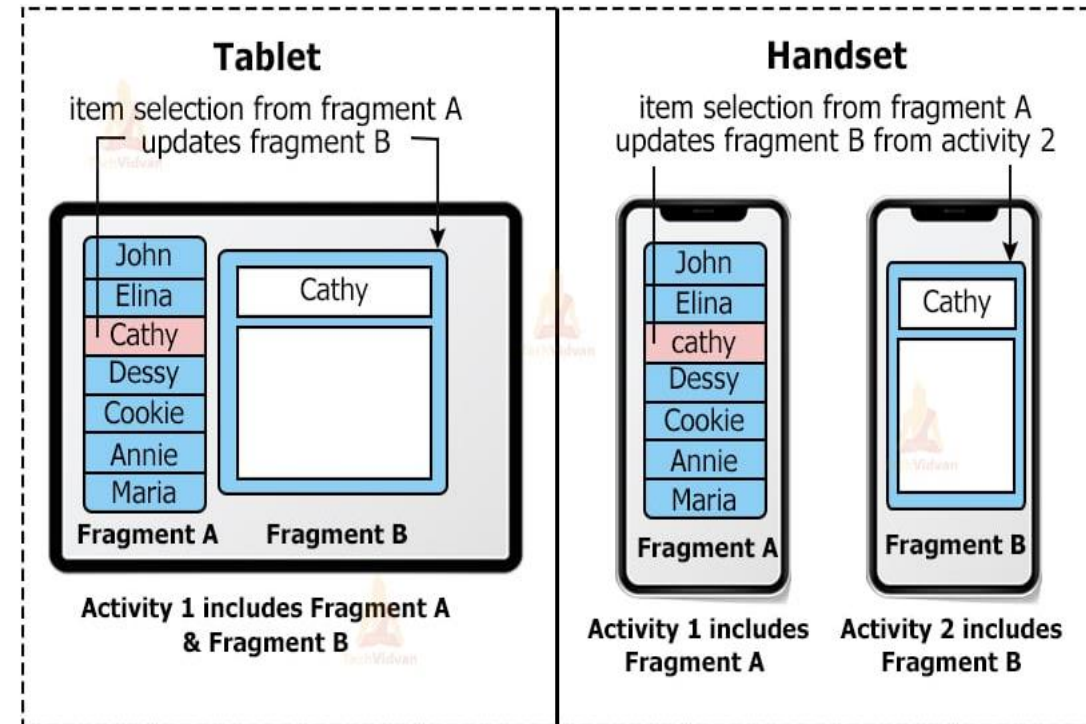
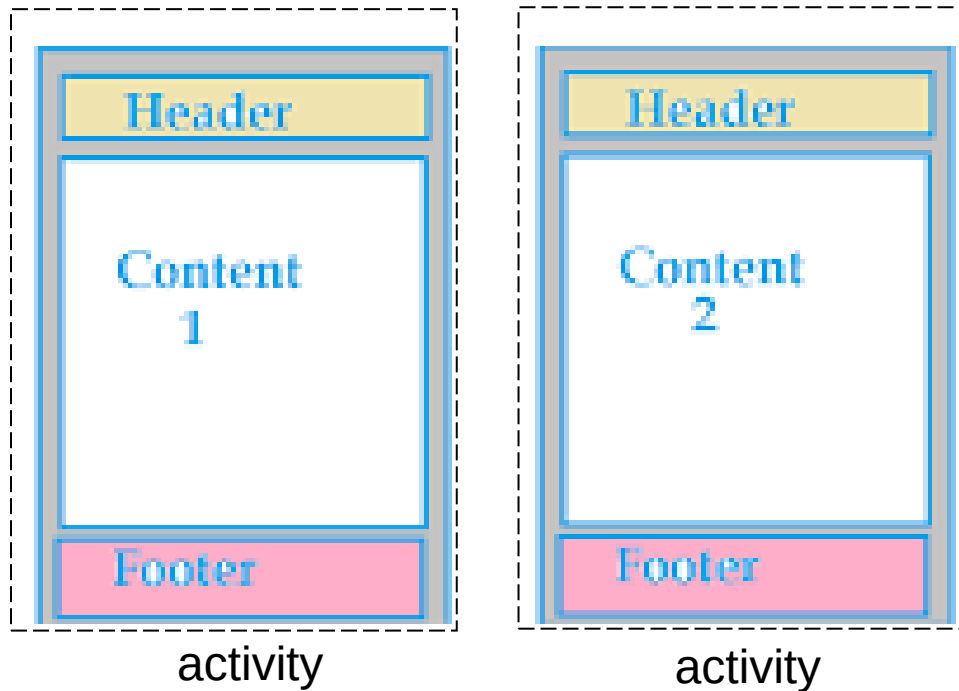
- **Advantages**

- Enables reusable UI components.
- Adapts easily to different devices (e.g., phones and tablets).
- Supports UI modularity.

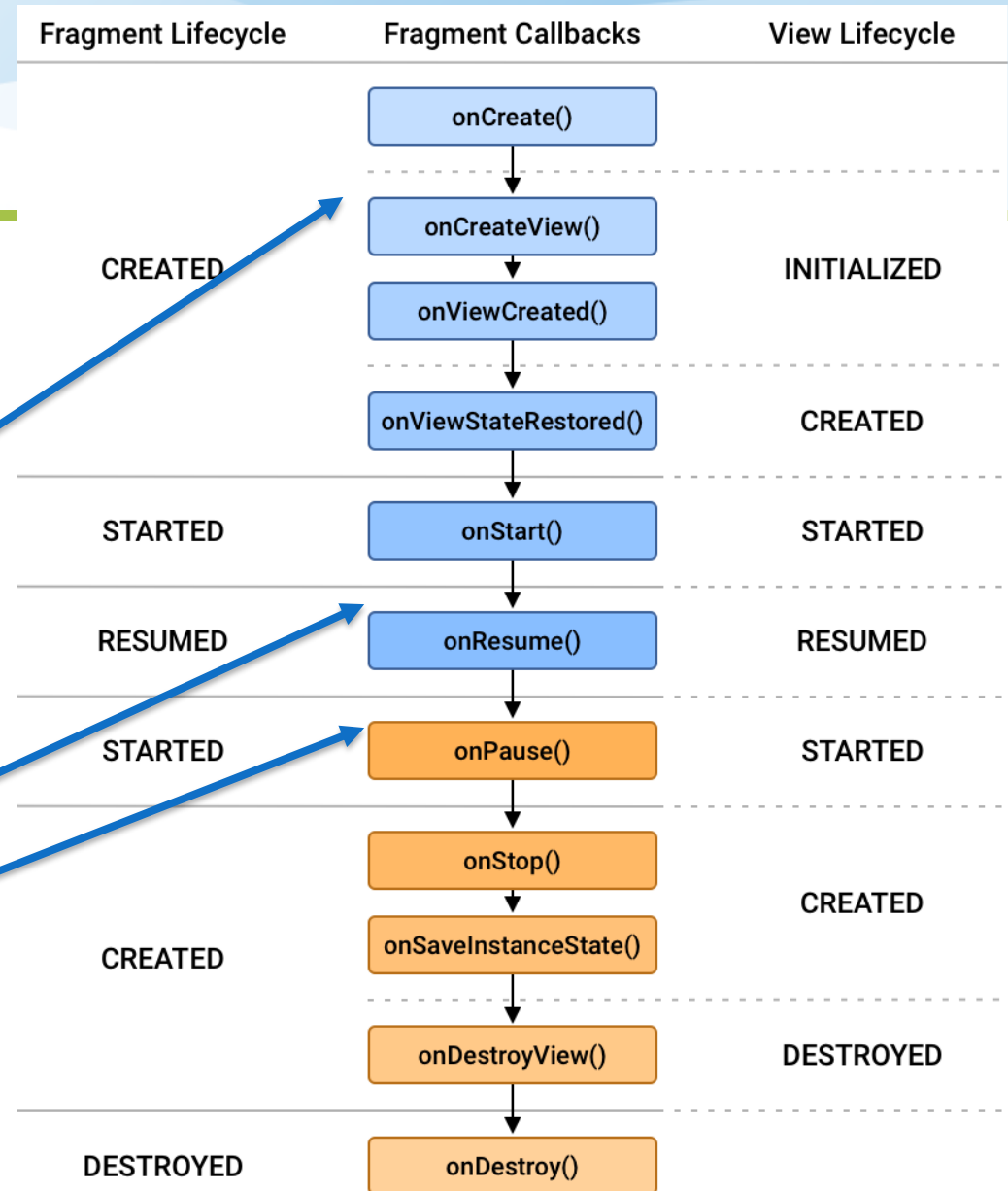
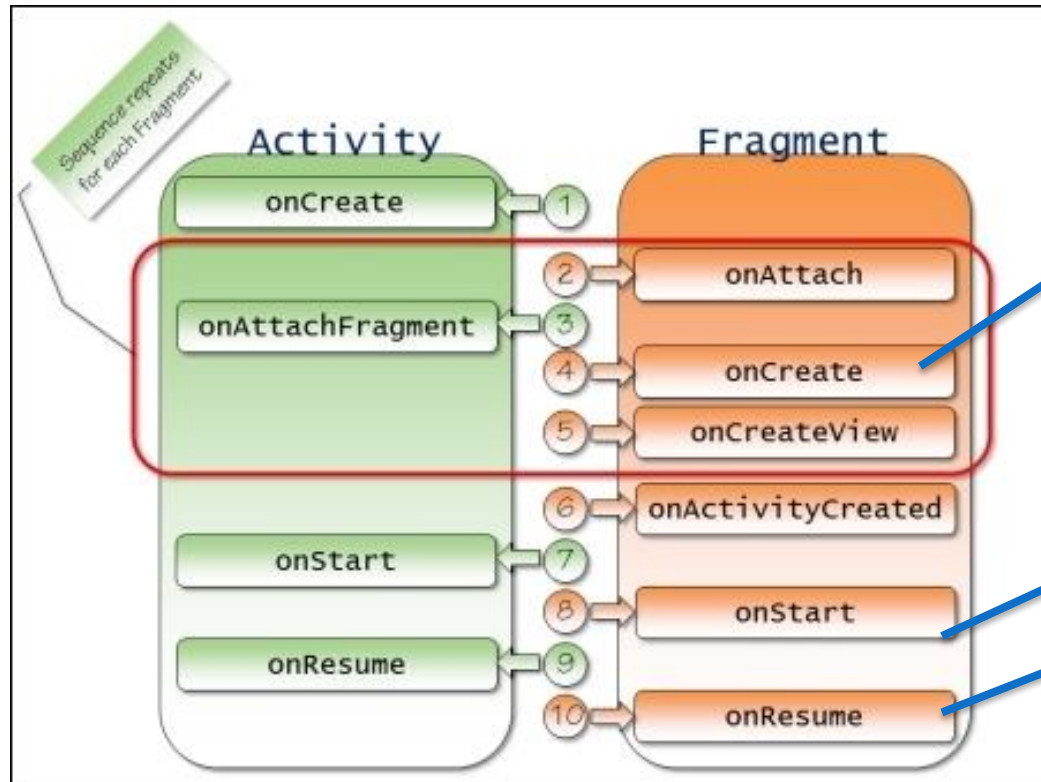


What is a Fragment? ..

- **Activity** is a container for views => typically fills the entire screen
- **Fragments** are introduced for Reusability, Adaptability & Modularity
 - One activity contains several mini-activities (fragments)



Fragment Lifecycle



Advantages of Using Fragments



- Modularity :
 - Diving complex activity code across fragment for better UI performance and maintenance.
- Reusability :
 - Placing behavior or UI parts into fragments that multiple activities can share.
- Adaptability :
 - Representing each sections of a UI as different fragment and utilizing the different layout on screen orientation or size.

Using: Add/Replace Fragments ?



1. add fragment **statically**

- Create a new Fragment
- Add a fragment tag inside activity and set name to the fragment and give id and ready for use it.

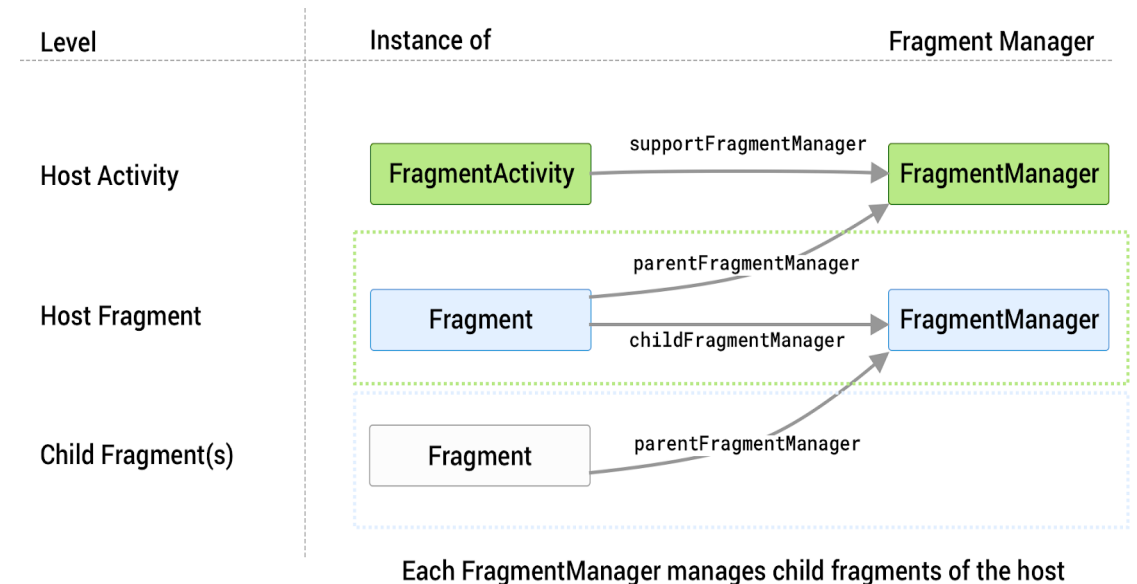
2. add fragment **dynamically**

□ Use **FragmentManager**

```
fragmentManager.beginTransaction()  
    .add(R.id.fragment_container, ExampleFragment())  
    .commit()
```

Common operations:

add(), replace(), remove().

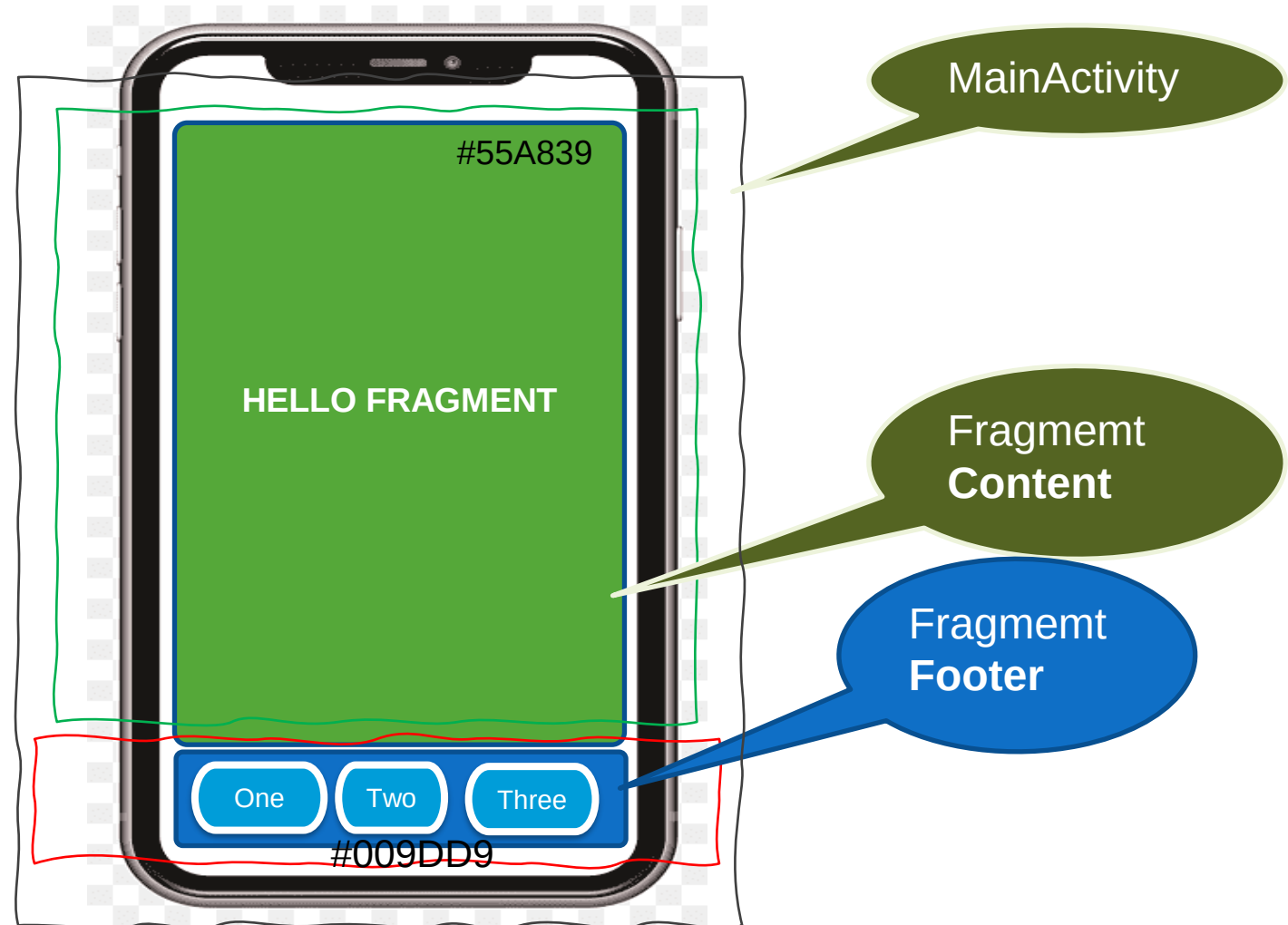


Add fragment statically: ex1

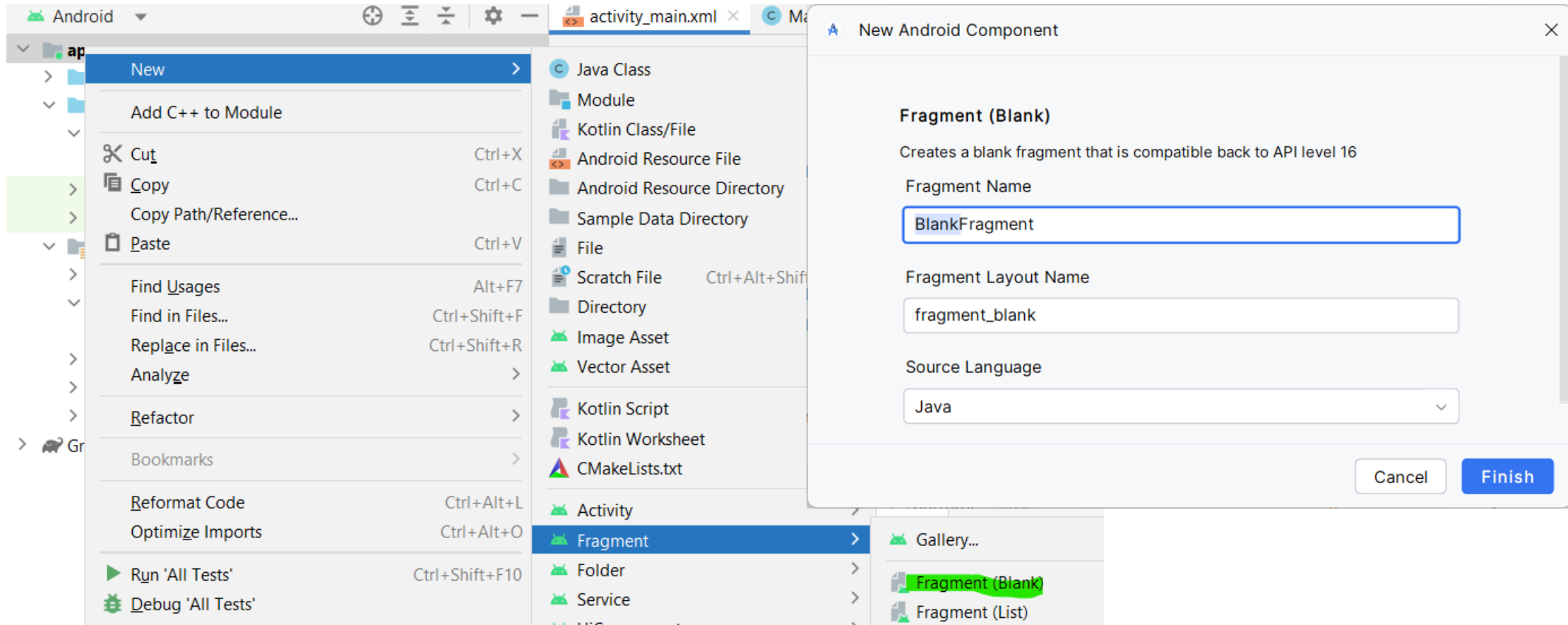


■ Step by step

- 1) Create a new Prj
- 2) Create ContentFragment and design
- 3) Create FooterFragment and design
- 4) Modify
 - 1) activity_main.xml layout to include 2 fragment
 - 2) Add two fragments using either a **FrameLayout** or **FragmentContainerView** to hold them (This approach allows future operations like **add**, **replace**, or **remove** fragments at runtime.)



Create new Fragment ?



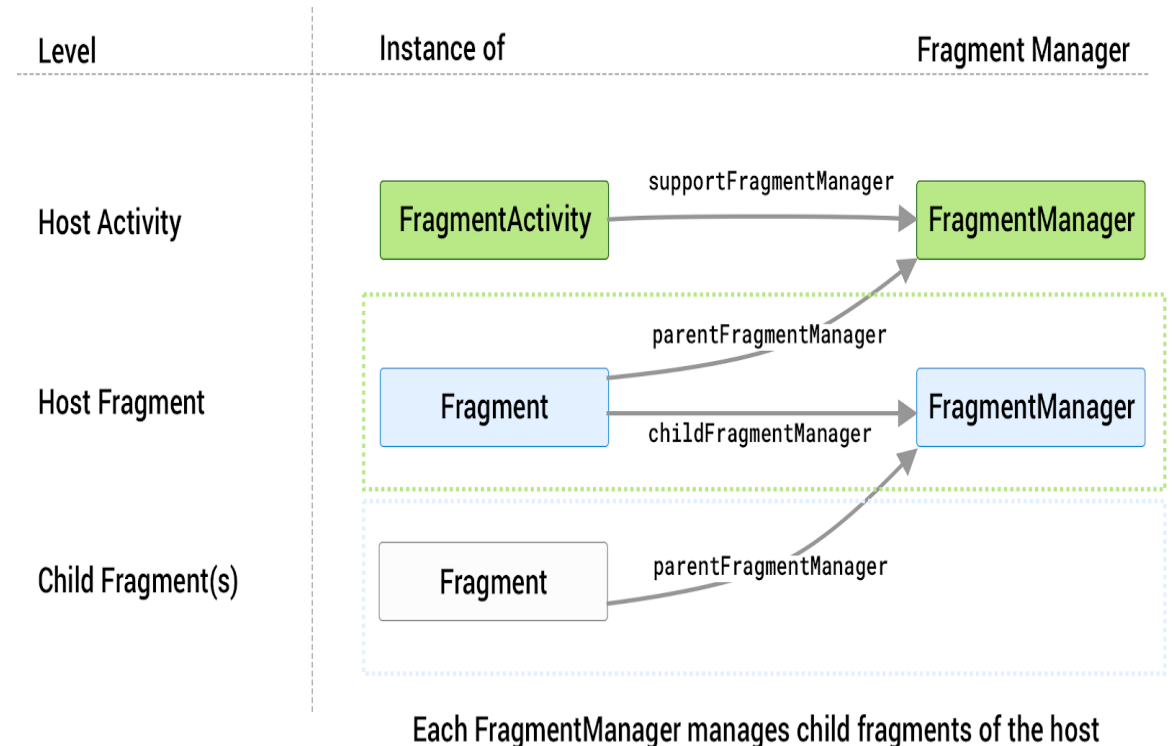
Adding Fragment Dynamically

■ FragmentManager

- is the class responsible for performing actions on your app's fragments, such as **adding**, **removing**, or **replacing** them and **adding** them to the **back stack**.

■ Access the **FragmentManager**

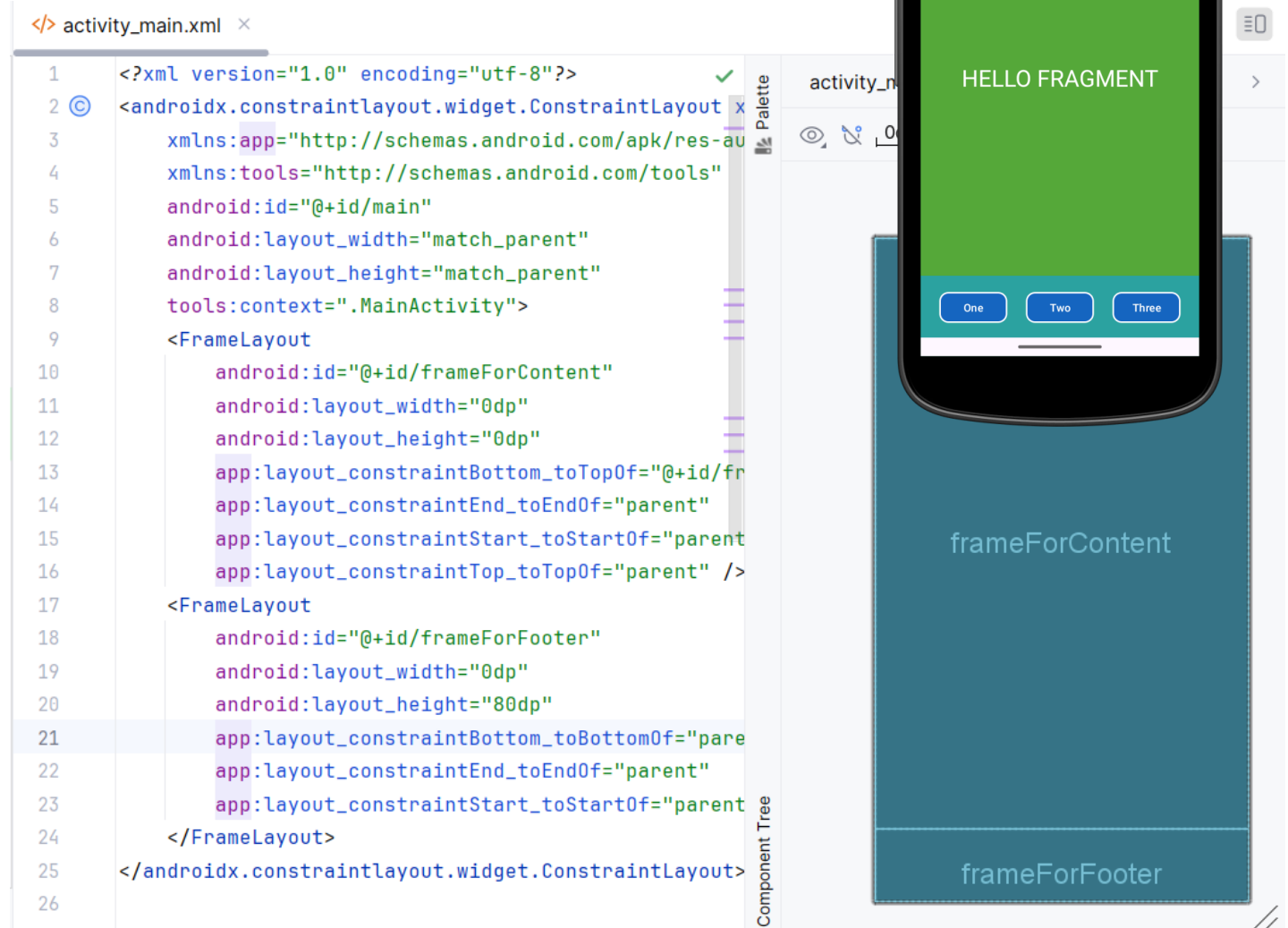
- From an activity
→ *getSupportFragmentManager()*
- From a fragment
→ *getParentFragmentManager()*



Adding Fragment Dynamically: Ex2

Step by step

- 1) Create a new Prj
- 2) Create ContentFragment and design
- 3) Create FooterFragment and design
- 4) Modify *activity_main.xml* layout to with 2 **placeholder** for 2 fragment, via **FrameLayout**
- 5) Code in MainActivity.java use **FragmentManager** class



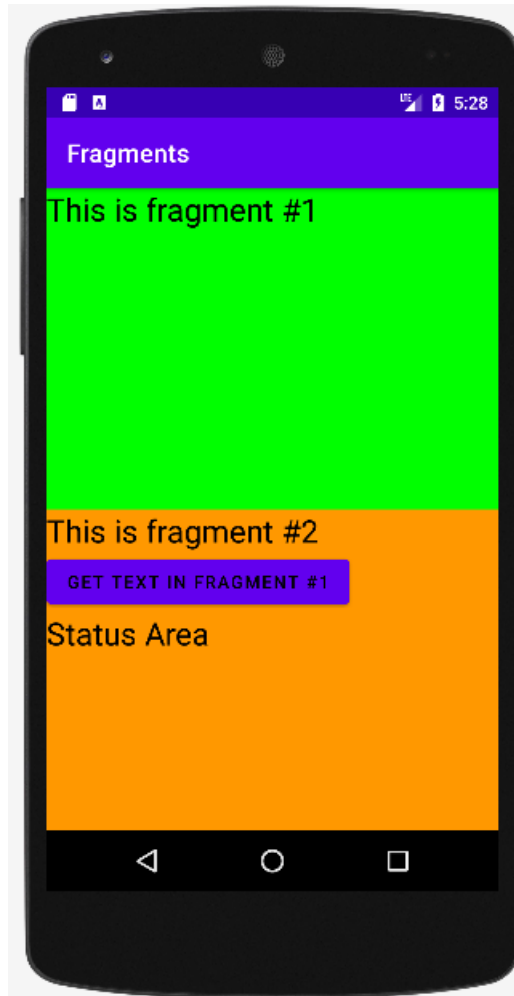
Replace Fragment Dynamically: Ex3

■ Step by step

- 1) Create a new Prj
- 2) Create *ContentFragment* and design
- 3) Create *FooterFragment* and design
- 4) Modify *activity_main.xml* layout to include 2 fragments, via **FragmentManager**
- 5) Create *FragmentOne*, *FragmentTwo*, *FragmentThree* and design
- 6) Code for Button *One*, *Two*, *Three*



Interactions between Fragments



Interactions between Fragments



- An activity might contain two or more fragments working together to present a coherent UI to the user
 - E.g.: the user taps on an item in that fragment, details about the selected item might be displayed in another fragment
- Continue in the same project, add bolded statements to **fragment_1.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:orientation="vertical" android:layout_width="fill_parent"
    android:layout_height="fill_parent" android:background="#00FF00"
    tools:context=".Fragment1">
    <TextView
        android:id="@+id/lblFragment1"
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="This is fragment #1"
        android:textColor="#000000"
        android:textSize="25sp" />
</LinearLayout>
```

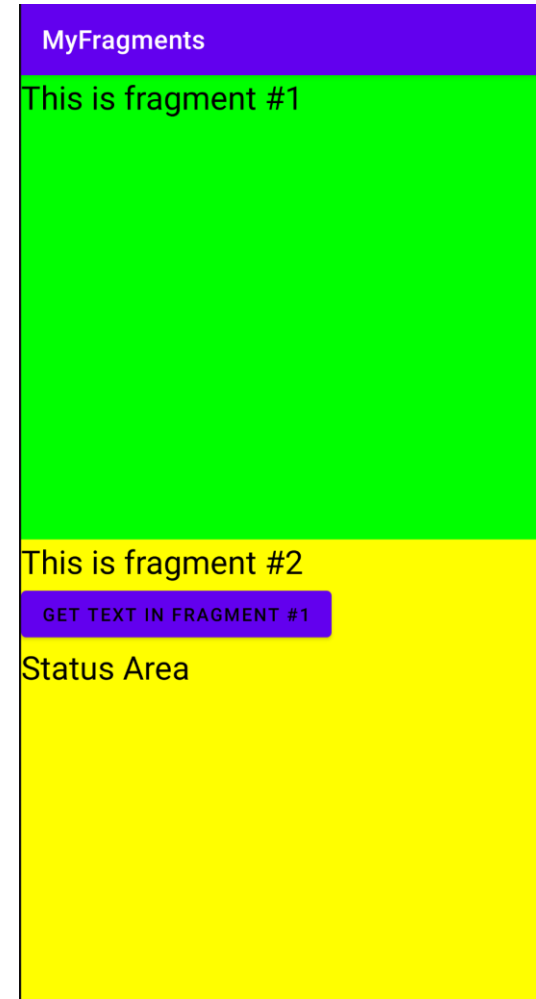
Interactions between Fragments



- Add the following bolded lines to fragment_2.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout

.....
<Button
    android:id="@+id/btnGetText"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Get text in Fragment #1"
    android:textColor="#000000"
    android:onClick="onClick" />
<TextView
    android:id="@+id/lblStatus"
    android:layout_width="fill_parent"
    android:layout_height="wrap_content"
    android:text="Status Area"
    android:textColor="#000000"
    android:textSize="25sp" />
</LinearLayout>
```



Interactions between Fragments



- In activity_main.xml, uncomment the two fragments:

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"    android:layout_height="match_parent"
    tools:context=".MainActivity"        android:orientation="vertical">
    <fragment
        android:name="com.maicuongtho.fragments.Fragment1"
        android:id="@+id/fragment1"
        android:layout_weight="1"
        android:layout_width="fill_parent"
        android:layout_height="match_parent" />
    <fragment
        android:name="com.maicuongtho.fragments.Fragment2"
        android:id="@+id/fragment2"
        android:layout_weight="1"
        android:layout_width="fill_parent"
        android:layout_height="match_parent" />
</LinearLayout>
```


Interactions between Fragments



- Modify the MainActivity.java file by commenting out the code added in the previous step, and **add setContentView() back**

```
package com.maicuongtho.fragments;
.....
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        //    fragmentManager fragmentManager = getSupportFragmentManager();
        //.....
        //    fragmentTransaction.commit();
    }
}
```

Interactions between Fragments



- Add the bolded statements to the Fragment2.java

Fragment2.java

```
...
public class Fragment2 extends Fragment {
...
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
        return inflater.inflate(R.layout.fragment_2, container, false);
    }
    @Override
    public void onStart() {
        super.onStart();
        Button btnGetText = (Button) getActivity().findViewById(R.id.btnGetText); //---Button view---
        btnGetText.setOnClickListener(new View.OnClickListener() { //---Button click---
            public void onClick(View v) {
                TextView lbl = (TextView) getActivity().findViewById(R.id.lblFragment1);
                TextView statusarea = (TextView) getActivity().findViewById(R.id.lblStatus);
                statusarea.setText("Fragment#1's text shows: "+lbl.getText());
            }
        });
    }
}
```

use a LayoutInflater object to inflate the UI from the specified XML file

Your turn

I fear not the man who has practiced 10,000 kicks once,
but I fear the man who has practiced one kick 10,000 times.
Bruce Lee



- Practice 7,8,9/ Exercice 7,8,9
 - 1) Repeat the examples by yourself (new project for each case)
 - 2) Push it to your github repository
 - ✓ With a report with screenshots of the final app in action, data structures used/class design, and the implementation logic.
- Practice 10/ Exercice 10 (Homework #4)

Homework #4



- Notepad app with custom keypad
 - Top fragment: display the note entered
 - Bottom fragment: display several rows of letters, numbers, and symbols (each as a button) for text input; when a user pushes a button, the corresponding input is appended in the top fragment

What to submit:

- Push it to your github repository,
 - a report with screenshots of the final app in action, data structures used/class design, and the implementation logic.
-